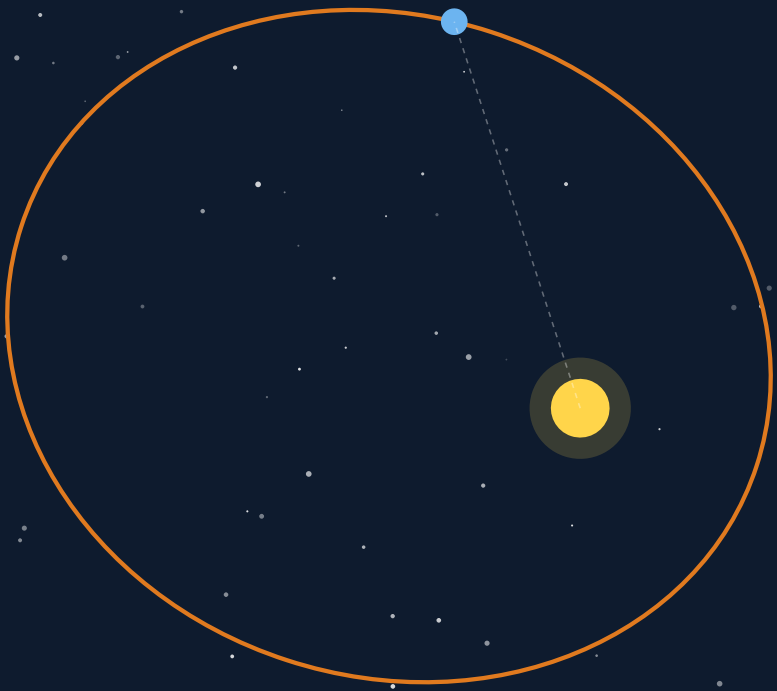


Rediscovering Kepler’s Three Laws

AN END-TO-END MACHINE LEARNING PROJECT

A step-by-step guide for complete beginners

Ellipses · Equal areas · Harmonies — found with linear regression

Before You Start

This guide walks you, step by step, through one complete Machine Learning project: **rediscovering Kepler's three laws of planetary motion from measurements**, the way a working data scientist would. You do not need to know any astronomy. You do need to be able to read a little Python, and to be curious.

It follows the teaching recipe of Chapter 2 (*End-to-End Machine Learning Project*) of Aurélien Géron's *Hands-On Machine Learning with Scikit-Learn and TensorFlow*: a story to give the work a purpose, a checklist to keep you on the path, small code cells you run yourself, a warning at every classic trap, and a "Try it out!" at the end. Where that chapter predicts house prices, we predict the motion of planets.

> **NOTE**
> **Everything here is reproducible.** Every number and figure in this PDF was produced by `kepler_project.py`, the companion script. Run it (about 10 seconds) and you will see the same output. The seeds are fixed; the last digit may differ with library versions.

> **WARNING**
> **About the data — please read.** The planet table (Law 3) holds real values, typed in from NASA/JPL reference tables and rounded. The Mars observations (Laws 1 and 2) are **simulated**: we start from Mars' real orbit, let Newton's gravity move it, and add measurement noise about the size of Tycho Brahe's. No Kepler law is built into the simulation. Exercise 5 shows how to swap in real data.

What you need

```
$ python3 -m pip install -U numpy pandas scipy scikit-learn matplotlib
$ python3 kepler_project.py      # runs the whole project, writes ./figures
```

Or paste the code cells from this guide, one at a time, into a Jupyter notebook. Python 3.9+ and a recent scikit-learn are enough.

Contents

Before You Start 2

Welcome to the Uraniborg Data Office 4

Look at the Big Picture 5

Frame the Problem 5

Select a Performance Measure 6
Check the Assumptions 6

Get the Data **7**

Create the Workspace 7
Download the Data (from a Fake Sky) 7
Take a Quick Look at the Data Structure 8
Create a Test Set 9

Law 1: The Orbit Is an Ellipse **11**

Discover and Visualize the Data to Gain Insights 11
Prepare the Data for Machine Learning Algorithms 13
Select and Train a Model 14
Fine-Tune Your Model 16
Evaluate Your System on the Test Set 17

Law 2: Equal Areas in Equal Times **19**

Discover and Visualize the Data to Gain Insights 19
Prepare the Data for Machine Learning Algorithms 20
Select and Train a Model 20
Fine-Tune Your Model: Which Law Breaks First? 23

Law 3: The Harmony of the Worlds **24**

Discover and Visualize the Data to Gain Insights 24
Prepare the Data for Machine Learning Algorithms 25
Select and Train a Model 25
Fine-Tune Your Model 27

Present Your Solution **28**

Launch, Monitor, and Maintain Your System **29**

Try It Out! **31**

Appendix: Kepler's Laws on One Page **32**

Welcome to the Uraniborg Data Office

In this guide you will go through an example project end to end, pretending to be a recently hired data scientist. The year is 1601. Tycho Brahe, the most precise observer of his age, has just died and left behind twenty years of careful naked-eye measurements of the planets. Your new boss, the mathematician Johannes Kepler, wants to know what rules the planets really obey.

Here are the main steps you will go through:

1. Look at the big picture.
2. Get the data.
3. Discover and visualize the data to gain insights.
4. Prepare the data for Machine Learning algorithms.
5. Select a model and train it.
6. Fine-tune your model.
7. Present your solution.
8. Launch, monitor, and maintain your system.

There is a twist. In the book's housing project, one question runs through all eight steps. Here we have **three** questions, one per law, so Steps 3 to 6 are run three times, each time a little faster than the last (Figure 1). By the third time you will do it almost without thinking, which is exactly the point.

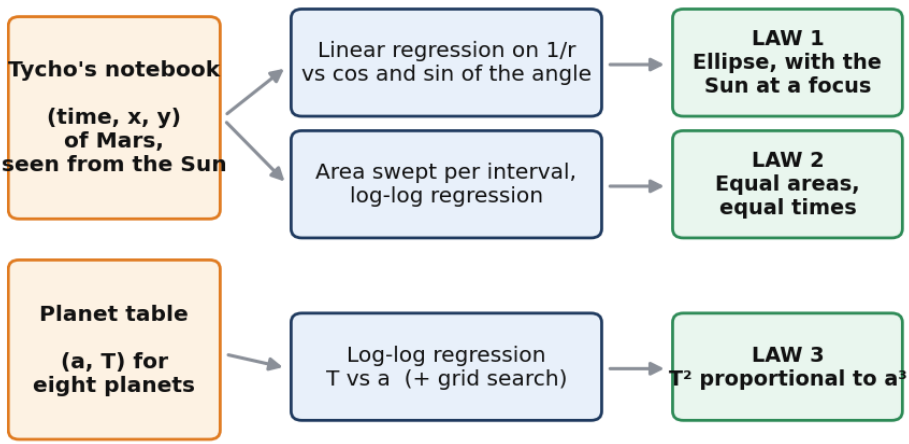


Figure 1. *The three mini-projects. Each one turns measurements into a law with a few lines of regression.*

> **A WORD ON HISTORY**
> The story is a dramatisation. The real Kepler did this by hand over years, and had to work out Mars' positions as seen from the Sun by clever triangulation. We start with those positions already in hand, so that we can focus on the Machine Learning workflow.

Look at the Big Picture

Welcome to the Uraniborg Data Office! Your first task is to find the rules behind the motion of Mars, and then to check whether the other planets obey the same rules. Mars is the best subject: its orbit is the most "non-circular" among the planets Tycho measured best.

Frame the Problem

The first question to ask your boss is what exactly the objective is. Kepler answers: *"I do not want a machine that merely spits out where Mars will be. The old tables already do that, roughly. I want the rules themselves: short statements, with a few numbers in them, that a person can read, check and reuse."*

This changes how you will work. A model that is only a black-box predictor is not good enough. Whatever you build must be **simple and readable**, with a handful of parameters that mean something. Keep that sentence in mind: it explains almost every modelling choice that follows.

The next question is what the current solution looks like. Kepler: the astronomers use circles, and circles upon circles (epicycles), and their best circle-based model misses Mars by about **8 arcminutes**, roughly a quarter of the width of the Full Moon. Tycho's instruments are accurate to about 1 to 2 arcminutes, so the old model cannot be right. That gap is your opportunity.

You are now ready to frame the problem. Before you read on, pause and try to answer these questions for yourself: Is this supervised, unsupervised, or Reinforcement Learning? Is it classification or regression? Should you use batch or online learning?

Have you found the answers? It is a typical **supervised** task, because every observation comes with the answer we want to explain (for instance, a distance from the Sun). It is a **regression** task, because we predict numbers. It is a **batch learning** task: no data streams in, and everything fits in memory. And you will keep your models **simple on purpose**.

TIP

Why not just use a Random Forest? You will, in fact, try one — and see two things. It can predict positions in the range it has seen, but it hands you no formula to give Kepler. And as soon as you ask it about a place it has not seen, it fails badly (Figures 7 and 12). For discovering laws, small readable models win.

Select a Performance Measure

A typical performance measure for regression is the Root Mean Square Error (RMSE). It tells you how much error the model typically makes, with extra weight on large errors (Equation 1).

$$RMSE(\mathbf{X}, h) = \sqrt{\left[(1/m) \sum_{i=1}^{m} \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2 \right]}$$

Equation 1. Root Mean Square Error

NOTATION

Notation. m is the number of observations you measure the error on. $\mathbf{x}^{(i)}$ holds the input features of the i-th observation and $y^{(i)}$ is its label, the answer you want. h is your model (the "hypothesis"): given $\mathbf{x}^{(i)}$ it outputs a prediction $\hat{y}^{(i)} = h(\mathbf{x}^{(i)})$.

Example. The first Mars observation has x = 1.266 AU and y = -0.551 AU, so its distance from the Sun is the label r = 1.381 AU. (An AU, or astronomical unit, is the Earth–Sun distance, about 150 million km.)

RMSE has the same unit as what you predict, which makes it easy to read: an RMSE of 0.001 AU means "typically off by 150,000 km". Two other rulers appear later. For Law 2 you will measure how much a quantity that *should* be constant actually wobbles (standard deviation divided by mean). For Law 3, where periods run from 0.24 to 165 years, plain errors would be dominated by Neptune, so you will use the **percentage error**.

Check the Assumptions

Lastly, list and verify the assumptions made so far, to catch serious issues early. We assume that: (1) positions are measured *from the Sun*, not from the Earth; (2) all motion happens in one flat plane (true to within a couple of degrees); (3) only the Sun matters, because the pull of other planets is tiny; (4) the clock is accurate and only the positions are noisy. You check with Kepler, he agrees, and the lights are green.

Get the Data

Create the Workspace

Install the libraries from the start of this guide, then either run `kepler_project.py` or open a Jupyter notebook and paste in the code cells that follow. Start with the imports:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)      # 42 again, as in the book
```

Download the Data (from a Fake Sky)

In the housing project you download a file. Our "telescope" is a function that simulates twenty-odd months of Mars observations. Do not worry if the details look new: what matters is the output, a table of *time, x, y* positions.

```
from scipy.integrate import solve_ivp

GM = 4 * np.pi ** 2        # the Sun's pull, in units of AU^3 / year^2

def sun_gravity(t, state):          # Newton: acceleration = -GM * position / r^3
    x, y, vx, vy = state
    r3 = (x * x + y * y) ** 1.5
    return [vx, vy, -GM * x / r3, -GM * y / r3]

def observe_mars(n_obs=300, years=3.76, noise=0.001, cloudy=0.08, seed=42):
    rng = np.random.default_rng(seed)
    # Mars' real orbit -- hidden from you until Step 7:
    a, e, omega = 1.52371, 0.0934, np.deg2rad(336.0)
    r_p = a * (1 - e)                          # start at the closest point
    v_p = np.sqrt(GM * (1 + e) / r_p)
    start = [r_p * np.cos(omega), r_p * np.sin(omega),
             -v_p * np.sin(omega), v_p * np.cos(omega)]
    gap = years / n_obs                        # about one look every 4-5 days
    t = np.arange(n_obs) * gap + rng.uniform(-0.3, 0.3, n_obs) * gap
    t = np.sort(np.clip(t, 0, years))
    sol = solve_ivp(sun_gravity, (0, years), start, t_eval=t,
                    method="DOP853", rtol=1e-11, atol=1e-12)
    x = sol.y[0] + rng.normal(0, noise, n_obs)    # every measurement is a bit off
    y = sol.y[1] + rng.normal(0, noise, n_obs)
    lost = rng.random(n_obs) < cloudy             # cloudy nights: no data at all
    x[lost], y[lost] = np.nan, np.nan
    return pd.DataFrame({"t": t, "x": x, "y": y})
```

> **NOTE**
>
> **No cheating.** The simulation uses only Newton's law of gravity (the acceleration of Mars points at the Sun and shrinks with the square of the distance). It contains no ellipse, no "equal areas" and no "period law". If you rediscover Kepler's laws from this data, it is because they really are in there. The orbit numbers in the code are hidden from your analysis; we reveal them in Step 7 to grade you.
>
> Each position is also nudged by random noise of 0.001 AU per coordinate. Seen from the Sun, that is about 2 arcminutes of sky: the same ballpark as Tycho's instruments.

Take a Quick Look at the Data Structure

```
>>> obs = observe_mars()
>>> obs.head()
          t         x         y
0  0.002060  1.266319 -0.551090
1  0.012074  1.287928 -0.498006
2  0.027763  1.317443 -0.418341
3  0.039084  1.336246 -0.356372
4  0.047082  1.349022 -0.311644
>>> obs.info()
<class 'pandas.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   t       300 non-null    float64
 1   x       283 non-null    float64
 2   y       283 non-null    float64
dtypes: float64(3)
memory usage: 7.2 KB
```

The table has 300 rows, one per observation, with the time t in years and the position (x, y) in AU. Notice that x and y have only 283 non-null values: 17 nights were cloudy and nothing was recorded. Both columns are empty in those rows, so there is nothing to repair. The simplest of the three options is to drop them.

```
>>> obs = obs.dropna()      # a cloudy night has nothing to repair: drop those rows
>>> len(obs)
283
>>> obs.describe()
                t           x           y
count  283.000000  283.000000  283.000000
mean     1.850923   -0.171440    0.088139
std      1.087050    1.071461    1.081271
min      0.002060   -1.652007   -1.460103
25%      0.908995   -1.210564   -0.998550
50%      1.842555   -0.235083    0.128749
75%      2.776703    0.892086    1.166897
max      3.737035    1.393071    1.575779
```

Notice a few things. The time runs from 0 to about 3.7 years, a little under two Mars years (one Mars year is 1.88 Earth years). More interestingly, x runs from -1.65 to 1.39: the cloud of positions is **not centred on the Sun**, which sits at (0, 0). Remember that.

Now the second dataset, for Law 3: the average distance a from the Sun (in AU) and the orbital period T (in years) of the eight planets.

```
>>> planets = pd.DataFrame({
...     "name": ["Mercury", "Venus", "Earth", "Mars",
...              "Jupiter", "Saturn", "Uranus", "Neptune"],
...     "a_au": [0.38710, 0.72334, 1.00000, 1.52371,
...              5.20289, 9.53668, 19.18916, 30.06992],
...     "T_years": [0.24085, 0.61520, 1.00002, 1.88082,
...                 11.8620, 29.4571, 84.0205, 164.79],
... })
>>> planets
      name      a_au    T_years
0  Mercury   0.38710    0.24085
1    Venus   0.72334    0.61520
2    Earth   1.00000    1.00002
3     Mars   1.52371    1.88082
4  Jupiter   5.20289   11.86200
5   Saturn   9.53668   29.45710
6   Uranus  19.18916   84.02050
7  Neptune  30.06992  164.79000
```

These values are taken from NASA/JPL reference tables and rounded; they are good enough for this project. Note how different the scales are: a runs from 0.39 to 30, and T from 0.24 to 165. The values are **tail-heavy**: most planets are bunched at small values and a few sit far to the right. When you meet that, think *logarithm*.

Create a Test Set

> **WAIT!**
> Lock away the test set **now**, before you look any further. Your brain is a powerful pattern detector, and if it spots a pattern in the test data it will quietly steer your modelling choices. Your final error estimate will then be too optimistic. This is called **data snooping bias**.

Each law needs a different kind of split.

Law 1: stratified sampling

For the orbit we put aside 20% of the observations. But a purely random split can, by bad luck, leave out part of the orbit. So we divide the sky into eight 45° sectors and make sure the test set has the right share from each (this is **stratified sampling**).

```
>>> obs["angle"] = np.arctan2(obs["y"], obs["x"])    # direction, seen from the Sun
>>> obs["sector"] = pd.cut(obs["angle"], bins=np.linspace(-np.pi, np.pi, 9),
...                        labels=range(1, 9))
>>> from sklearn.model_selection import StratifiedShuffleSplit, train_test_split
>>> split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
>>> for train_idx, test_idx in split.split(obs, obs["sector"]):
...     train_set, test_set = obs.iloc[train_idx], obs.iloc[test_idx]
>>> len(train_set), len(test_set)
(226, 57)

def sector_props(data):
    return data["sector"].value_counts() / len(data)
```

```
_, random_test = train_test_split(obs, test_size=0.2, random_state=42)
compare = pd.DataFrame({"Overall": sector_props(obs),
                        "Stratified": sector_props(test_set),
                        "Random": sector_props(random_test)}).sort_index()
compare["Rand. %error"] = 100 * compare["Random"] / compare["Overall"] - 100
compare["Strat. %error"] = 100 * compare["Stratified"] / compare["Overall"] - 100
print(compare.round(3))
```

```
        Overall  Stratified  Random  Rand. %error  Strat. %error
sector
1         0.141       0.140   0.088       -37.939         -0.702
2         0.124       0.123   0.123        -0.702         -0.702
3         0.110       0.105   0.123        12.111         -3.905
4         0.102       0.105   0.088       -14.398          2.722
5         0.117       0.123   0.140        20.362          5.316
6         0.127       0.123   0.105       -17.251         -3.460
7         0.141       0.140   0.105       -25.526         -0.702
8         0.138       0.140   0.228        65.497          1.844
```

Figure 2 shows the same numbers. A purely random test set is skewed: sector 8 is over-represented by 65% and sector 1 under-represented by 38%. The stratified one is within 5% everywhere.

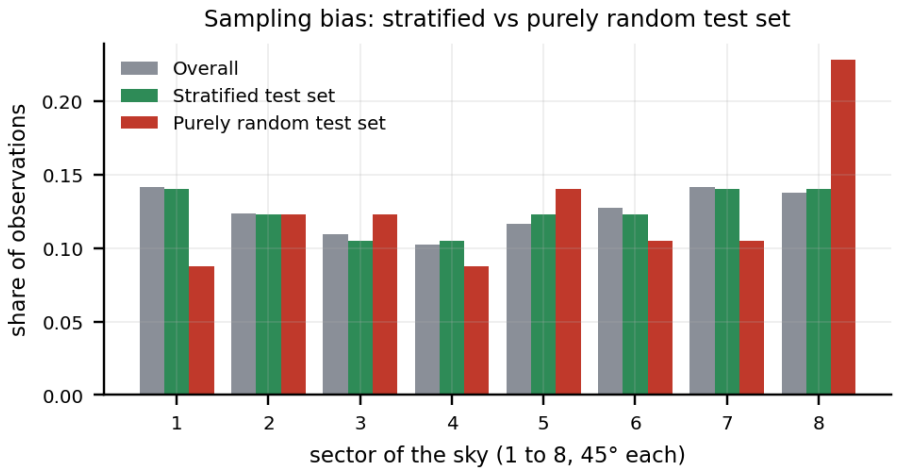


Figure 2. *Sampling bias: stratified versus purely random test set.*

Law 3: lock away the outermost planets

With only eight planets, a random 20% is one or two planets picked by luck. We do something smarter: train on the **six planets known in Kepler's day** (Mercury to Saturn), and lock away **Uranus** (discovered in 1781) and **Neptune** (1846). This tests something stronger than a random split would. It tests **extrapolation**: can the law predict worlds that are far beyond anything it was trained on?

```
>>> train_planets = planets.iloc[:6]     # the six planets known in Kepler's day
>>> test_planets = planets.iloc[6:]      # Uranus and Neptune: locked away
>>> test_planets["name"].tolist()
['Uranus', 'Neptune']
```

Law 2: no test set needed

Law 2 has almost nothing to overfit: we will measure one quantity, not tune a flexible model. It also needs observations in time order, so we cannot scatter them into random piles. We use the whole record. Since Law 2 never influences how we build the Law 1 model, the Law 1 test set stays clean. (If you are a purist, run Law 2 on `train_set` alone; the result is the same.)

Law 1: The Orbit Is an Ellipse

Kepler's first law: every planet moves on an ellipse, with the Sun at one of its two foci. In ML terms: find a simple formula for the distance r from the Sun as a function of the direction (the angle θ) in which we see the planet.

Discover and Visualize the Data to Gain Insights

Make sure you have put the test set aside and are only exploring the training set. Let us also make a copy so we can play with it without harming it.

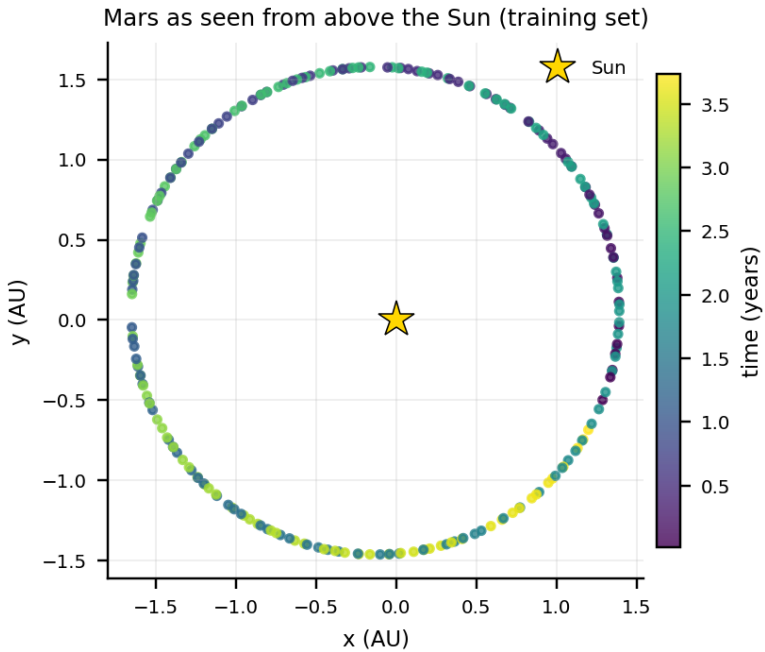


Figure 3. *Mars as seen from above the Sun. Colour shows time; the cloud wraps around twice.*

It looks like a circle, but the Sun is visibly off-centre. Let us compute each point's distance from the Sun and look at its summary.

```
orbit = train_set.copy()
orbit["r"] = np.hypot(orbit["x"], orbit["y"])       # distance from the Sun (AU)
```

```
orbit["cos_th"] = np.cos(orbit["angle"])
orbit["sin_th"] = np.sin(orbit["angle"])
print(orbit["r"].describe().round(3))
print(orbit[["r", "angle", "cos_th", "sin_th"]].corr()["r"].round(3))
```

```
count    226.000
mean       1.529
std        0.099
min        1.380
25%        1.433
50%        1.534
75%        1.625
max        1.667
Name: r, dtype: float64
r         1.000
angle     0.332
cos_th   -0.907
sin_th    0.401
Name: r, dtype: float64
```

The distance *r* ranges from 1.38 to 1.67 AU. So Mars is not at a constant distance from the Sun: a plain circle around the Sun is wrong. The correlation table needs care. The raw angle correlates only weakly with *r* (0.33). But the angle *wraps around*: 179° and −179° are neighbours, and a raw angle thinks they are far apart. Using cos θ and sin θ (the x and y coordinates of the direction on a unit circle) avoids that, and the correlation with cos θ jumps to −0.91.

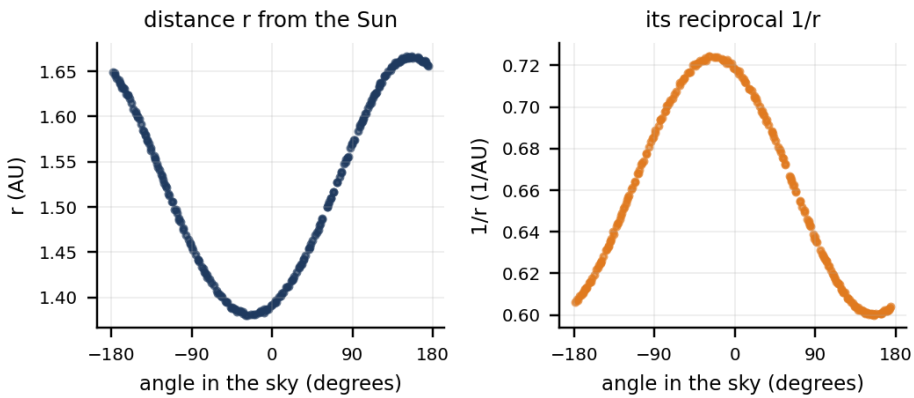


Figure 4. *Distance r (left) and its reciprocal 1/r (right) against the viewing angle. Both look like one clean wave.*

Experimenting with attribute combinations

Correlation only measures *straight-line* relationships between two things, so for several features at once we use a cousin: $\mathbf{R^2}$, the share of the target's ups and downs that the model explains (1.0 is perfect). Let us compare feature sets, and also try predicting the *reciprocal* 1/*r* instead of *r* (why? see the Math Note below).

```
from sklearn.linear_model import LinearRegression

tests = {"angle only": ["angle"],
         "cos(angle) only": ["cos_th"],
```

```
         "cos and sin": ["cos_th", "sin_th"]}
for label, cols in tests.items():
    for target, y in {"r": orbit["r"], "1/r": 1 / orbit["r"]}.items():
        R2 = LinearRegression().fit(orbit[cols], y).score(orbit[cols], y)
        print(f"{label:16s} -> predict {target:3s}: R^2 = {R2:.4f}")
```

```
angle only       -> predict r  : R^2 = 0.1104
angle only       -> predict 1/r: R^2 = 0.1009
cos(angle) only  -> predict r  : R^2 = 0.8228
cos(angle) only  -> predict 1/r: R^2 = 0.8252
cos and sin      -> predict r  : R^2 = 0.9977
cos and sin      -> predict 1/r: R^2 = 0.9999
```

Now we are getting somewhere. The angle alone explains little ($R^2 \approx 0.11$). Cos and sin together explain 99.8% of the variation of r — and a stunning **99.99%** of the variation of $1/r$. The reciprocal is clearly the more natural quantity.

> **MATH NOTE**
>
> A conic section (ellipse, parabola or hyperbola) with the Sun at a focus has the polar equation
>
> $r(\theta) = p \,/\, (1 + e\cos(\theta - \omega))$, where $p = a(1 - e^2)$.
>
> Flip it upside down: $1/r = 1/p + (e/p)\cos\omega \cdot \cos\theta + (e/p)\sin\omega \cdot \sin\theta$, which has the form $\mathbf{1/r = A + B\cos\theta + C\sin\theta}$. That is just a **straight-line model** in the features $\cos\theta$ and $\sin\theta$! Once you fit A, B and C, you can read off the shape: $p = 1/A$, $e = \sqrt{(B^2+C^2)}/A$ and $\omega = \text{atan2}(C, B)$. If $e < 1$ the orbit is an ellipse ($a = p/(1-e^2)$).
>
> *(We take this polar formula from geometry as given; Exercise 6 asks you to derive it.)*

Prepare the Data for Machine Learning Algorithms

Instead of doing this by hand, write a small class that does it for you. That lets you reuse it, apply it to fresh data later, and plug it into a pipeline. Scikit-Learn needs just three methods: `fit`, `transform` and (for free, via a mixin) `fit_transform`.

```
from sklearn.base import BaseEstimator, TransformerMixin

class OrbitFeatures(BaseEstimator, TransformerMixin):
    """(x, y) seen from the Sun -> cos(k*angle), sin(k*angle), k = 1..n_harmonics"""
    def __init__(self, n_harmonics=1):
        self.n_harmonics = n_harmonics
    def fit(self, X, y=None):
        return self                                  # nothing to learn
    def transform(self, X):
        X = np.asarray(X)
        theta = np.arctan2(X[:, 1], X[:, 0])
        cols = []
        for k in range(1, self.n_harmonics + 1):
            cols += [np.cos(k * theta), np.sin(k * theta)]
        return np.column_stack(cols)

X_train = train_set[["x", "y"]].to_numpy()
r_train = np.hypot(train_set["x"], train_set["y"]).to_numpy()
X_test = test_set[["x", "y"]].to_numpy()
r_test = np.hypot(test_set["x"], test_set["y"]).to_numpy()
```

The class has one **hyperparameter**, `n_harmonics`, which adds cos 2θ, sin 2θ and so on. It defaults to 1 (the pure ellipse). As in the book, a hyperparameter that gates a step you are not sure about lets you test it automatically later.

> **TIP**
> **Feature scaling?** Normally you must put features on the same scale. Here cos and sin already live in [−1, 1], so no scaler is needed. **Cleaning?** The only gaps, the cloudy nights, were dropped in Step 2.

Select and Train a Model

Let us try five models, from too simple to too flexible, all with the same pipeline structure. The Sun-focus ellipse uses a `TransformedTargetRegressor`: it fits a straight line to 1/*r* and flips the answer back into *r*, so every model is scored on the same ruler.

```
from sklearn.pipeline import make_pipeline
from sklearn.dummy import DummyRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.compose import TransformedTargetRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import cross_val_score

def make_ellipse_model(n_harmonics=1):
    # fit a straight line to 1/r, then flip the prediction back into r
    return TransformedTargetRegressor(
        regressor=make_pipeline(OrbitFeatures(n_harmonics), LinearRegression()),
        func=np.reciprocal, inverse_func=np.reciprocal)

models = {
    "Circle around the Sun": make_pipeline(OrbitFeatures(), DummyRegressor()),
    "Off-centre circle (r)": make_pipeline(OrbitFeatures(), LinearRegression()),
    "Decision tree": make_pipeline(OrbitFeatures(),
                                   DecisionTreeRegressor(random_state=42)),
    "Random forest": make_pipeline(OrbitFeatures(),
                                   RandomForestRegressor(random_state=42)),
    "Sun-focus ellipse (1/r)": make_ellipse_model(),
}

rows = []
for name, model in models.items():
    model.fit(X_train, r_train)
    train_rmse = np.sqrt(mean_squared_error(r_train, model.predict(X_train)))
    scores = cross_val_score(model, X_train, r_train, cv=10,
                             scoring="neg_mean_squared_error")
    cv = np.sqrt(-scores)
    rows.append((name, train_rmse, cv.mean(), cv.std()))
results1 = pd.DataFrame(rows, columns=["model", "train RMSE", "CV RMSE", "CV std"])
results1 = results1.set_index("model")
print(results1.round(5))
```

```
                        train RMSE  CV RMSE   CV std
model
Circle around the Sun       0.09906  0.09915  0.00698
Off-centre circle (r)       0.00474  0.00480  0.00029
Decision tree               0.00000  0.00256  0.00040
Random forest               0.00071  0.00198  0.00044
```

```
Sun-focus ellipse (1/r)     0.00100  0.00101  0.00011
```

Read the table the way the book teaches you to:

- **Circle around the Sun** (RMSE 0.099 AU) is **underfitting**. It ignores the angle completely. That error is 100 times the noise.
- **Off-centre circle** (0.0048 AU) is the Copernican/Ptolemaic idea of a circle shifted away from the Sun. It is far better, yet still 5 times the noise. 0.0048 AU seen from the Sun is about 11 arcminutes, the same ballpark as Kepler's famous "eight minutes" that made him abandon circles.
- **Decision tree** has a training error of exactly **0.0**. Zero error would be suspicious in any real measurement, and indeed the tree has memorised the noise: its cross-validated error is 0.0026 AU. That is **overfitting**.
- **Random forest** does better (0.0020 AU) but still shows a gap between training (0.00071) and validation.
- **Sun-focus ellipse** has train 0.00100 and CV 0.00101 AU — equal, and equal to the measurement noise (0.001 AU). There is nothing left to learn.

> **NOTE**
>
> **Cross-validation** (CV). The training set is shuffled and split into 10 piles (folds). The model is trained on 9 piles and scored on the 10th; this is repeated 10 times, so every observation is predicted once by a model that never saw it. The mean tells you how good the model is; the standard deviation tells you how sure you can be. Scikit-Learn's scorers prefer "higher is better", so the code negates the squared error, hence `neg_mean_squared_error` and the minus sign before the square root.

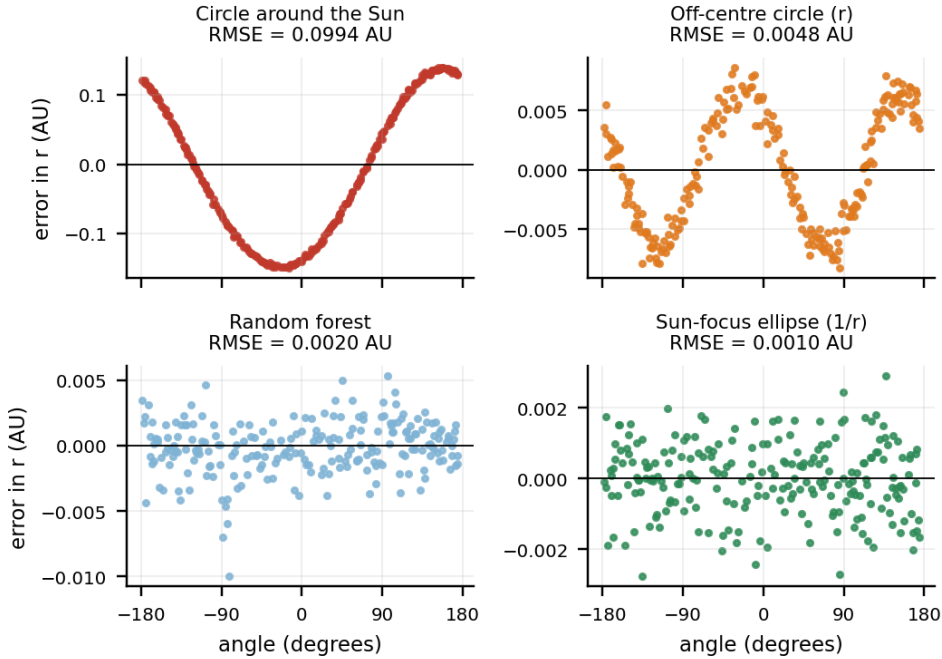


Figure 5. *Errors of four models, by viewing angle. The first three show structure left over; the last is pure noise.*

Figure 5 is the most important plot of this project. A good model leaves residuals that look like **featureless noise**. The circle leaves a giant wave. The off-centre circle leaves a smaller, faster wave (twice per lap) that the ellipse formula explains. The forest is noisy and spiky. The ellipse leaves nothing.

Fine-Tune Your Model

Grid search

Is the plain ellipse enough, or does the orbit need extra wiggles (more harmonics)? You could try by hand, but Scikit-Learn's `GridSearchCV` will evaluate every value of the hyperparameter with cross-validation. The name `regressor__orbitfeatures__n_harmonics` spells out the path to the knob: outer model → pipeline step → parameter.

```
from sklearn.model_selection import GridSearchCV

param_grid = {"regressor__orbitfeatures__n_harmonics": [1, 2, 3, 4]}
grid_search = GridSearchCV(make_ellipse_model(), param_grid, cv=10,
                           scoring="neg_mean_squared_error")
grid_search.fit(X_train, r_train)

scores = grid_search.cv_results_["mean_test_score"]
for n, score in zip(param_grid["regressor__orbitfeatures__n_harmonics"], scores):
    print(f"harmonics = {n}:  CV RMSE = {np.sqrt(-score):.6f} AU")
```

```
harmonics = 1:  CV RMSE = 0.001018 AU
harmonics = 2:  CV RMSE = 0.001027 AU
harmonics = 3:  CV RMSE = 0.001033 AU
harmonics = 4:  CV RMSE = 0.001024 AU
```

All four cross-validated errors agree to within 0.00002 AU, far smaller than the fold-to-fold spread (0.00011 AU). Extra harmonics buy nothing: the single harmonic is a complete description. We follow **Occam's razor** and keep the simplest model.

How sure are you? Bootstrap

Predictions are one thing. Kepler wants numbers with error bars. We read the orbit's **semi-major axis** a (half the long diameter), the **eccentricity** e (0 = circle, near 1 = very squashed) and the direction ω of closest approach from the three fitted numbers, and use the **bootstrap**: refit the model on 1,000 re-sampled copies of the training data, and see how much the answers move.

```python
def orbital_elements(model):
    lin = model.regressor_[-1]              # the LinearRegression inside
    A, (B, C) = lin.intercept_, lin.coef_   # 1/r = A + B cos(angle) + C sin(angle)
    p = 1 / A                               # the "semi-latus rectum"
    e = np.hypot(B, C) / A                  # eccentricity
    omega = np.degrees(np.arctan2(C, B)) % 360     # direction of closest approach
    a = p / (1 - e ** 2)                    # semi-major axis
    return a, e, omega

from sklearn.base import clone
rng = np.random.default_rng(42)
boot = []
for _ in range(1000):                       # bootstrap: refit on re-sampled data
    idx = rng.integers(0, len(X_train), len(X_train))
    refit = clone(final_model).fit(X_train[idx], r_train[idx])
    boot.append(orbital_elements(refit))
lo, hi = np.percentile(boot, [2.5, 97.5], axis=0)

names = ["a (AU)", "e", "omega (deg)"]
for name, value, l, h in zip(names, orbital_elements(final_model), lo, hi):
    print(f"{name:12s} = {value:8.4f}   95% CI [{l:.4f}, {h:.4f}]")
```

```
a (AU)       =   1.5239   95% CI [1.5237, 1.5240]
e            =   0.0934   95% CI [0.0933, 0.0935]
omega (deg)  = 336.0000   95% CI [335.9339, 336.0680]
```

Evaluate Your System on the Test Set

Now the moment of truth. Open the vault, run the final model on the test set, and, as in the book, compute a 95% confidence interval for the error.

```python
# Occam's razor: the simplest setting (1 harmonic) is as good as any other
final_model = make_ellipse_model(n_harmonics=1).fit(X_train, r_train)

from scipy import stats
final_pred = final_model.predict(X_test)
squared_errors = (final_pred - r_test) ** 2
final_rmse = np.sqrt(squared_errors.mean())
ci = np.sqrt(stats.t.interval(0.95, len(squared_errors) - 1,
                              loc=squared_errors.mean(),
                              scale=stats.sem(squared_errors)))
print(f"Test RMSE = {final_rmse:.5f} AU,  95% CI = [{ci[0]:.5f}, {ci[1]:.5f}]")
```

```
Test RMSE = 0.00099 AU,  95% CI = [0.00078, 0.00117]
```

The test RMSE is 0.00099 AU, almost exactly the noise we injected (0.001 AU). The test error is no worse than the cross-validation error, so we did not fool ourselves while tuning. Figure 6 shows the result: the fitted ellipse, with the Sun at one focus and a second, empty focus. The orange test points sit on the curve.

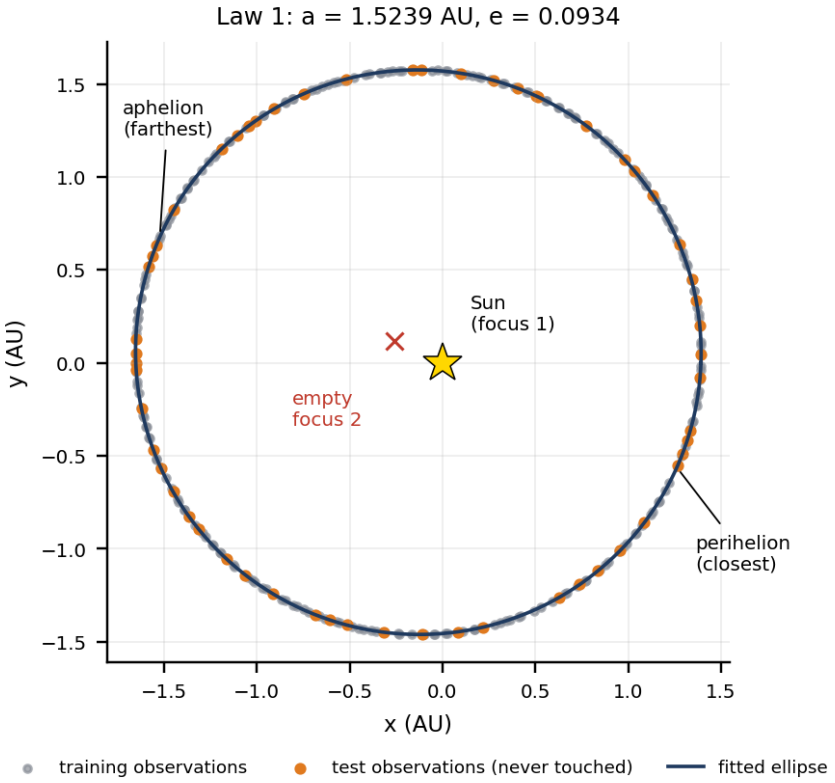


Figure 6. *Law 1: the fitted ellipse. The Sun sits at a focus, not at the centre. Orange points were never used for fitting.*

A stress test: can it extrapolate?

A last check that mirrors the Law 3 idea. Train on only the lower half of the orbit (angles below 0°), and predict the half that the model never saw.

```
# Stress test: learn from HALF the orbit, predict the OTHER half
theta_train = np.arctan2(X_train[:, 1], X_train[:, 0])
seen = theta_train < 0
extrap = {}
for name in ["Decision tree", "Random forest",
             "Off-centre circle (r)", "Sun-focus ellipse (1/r)"]:
    model = clone(models[name]).fit(X_train[seen], r_train[seen])
    pred = model.predict(X_train[~seen])
    extrap[name] = np.sqrt(mean_squared_error(r_train[~seen], pred))
    print(f"{name:24s} RMSE on the unseen half: {extrap[name]:.4f} AU")
```

```
Decision tree            RMSE on the unseen half: 0.0682 AU
Random forest            RMSE on the unseen half: 0.0747 AU
Off-centre circle (r)    RMSE on the unseen half: 0.0146 AU
Sun-focus ellipse (1/r)  RMSE on the unseen half: 0.0011 AU
```

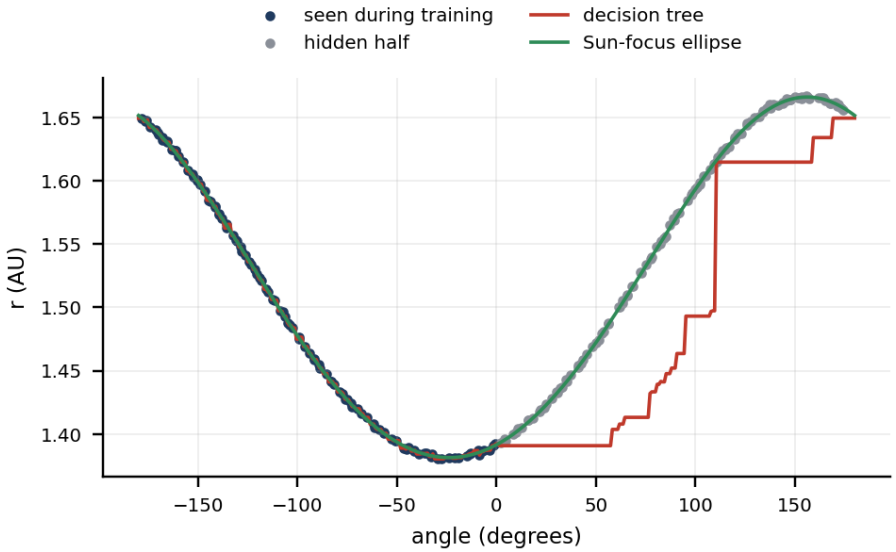


Figure 7. *Learning from half the orbit. The tree repeats its last value; the ellipse continues correctly.*

The tree and forest are off by about 0.07 AU. The off-centre circle is off by 0.015. The ellipse is off by 0.0011 AU — the noise level, even though it saw only half the orbit. **A model that captures the law extrapolates; a model that memorises cannot.**

Law 2: Equal Areas in Equal Times

Kepler's second law: the line from the Sun to a planet sweeps out equal areas in equal times. So a planet moves quicker when it is near the Sun and slower when it is far. We shall *discover* that, not assume it.

Discover and Visualize the Data to Gain Insights

Which quantity stays constant as Mars moves? Three candidates come to mind, each from a different era of astronomy:

- **Speed** (distance travelled per year): the ancient idea that heavenly motion is uniform.
- **Angular velocity** (how fast the viewing angle changes): the idea behind the Ptolemaic "equant".

- **Areal velocity** (area swept out per year): Kepler's guess.

Prepare the Data for Machine Learning Algorithms

All three can be computed from *neighbouring* observations. For two positions seen one after the other, the Sun and the two positions form a thin triangle whose area is half a cross product.

> **MATH NOTE**
> The triangle with corners (0, 0), (x_1, y_1) and (x_2, y_2) has area ½ $(x_1 y_2 - x_2 y_1)$. Divide by the elapsed time and you have the areal velocity. For the angular velocity we use `np.unwrap`, which removes the artificial jump from +180° to −180° each time the planet completes a lap.

Select and Train a Model

Here the "model" is a hypothesis ("this quantity is constant") and the ruler is how much it wobbles: the standard deviation divided by the mean.

```
track = obs.sort_values("t")           # all rows, in time order
t = track["t"].to_numpy()
x, y = track["x"].to_numpy(), track["y"].to_numpy()

dt = np.diff(t)                        # years between neighbouring looks
# area of the triangle: Sun + two neighbouring positions
area = 0.5 * (x[:-1] * y[1:] - x[1:] * y[:-1])
areal_velocity = area / dt             # area swept per year (AU^2 / yr)
speed = np.hypot(np.diff(x), np.diff(y)) / dt             # AU / yr
ang_vel = np.diff(np.unwrap(np.arctan2(y, x))) / dt       # radians / yr
r = np.hypot(x, y)
r_mid = 0.5 * (r[:-1] + r[1:])         # distance during each interval

quantities = {"speed": speed, "angular velocity": ang_vel,
              "areal velocity": areal_velocity}
spread = pd.DataFrame({
    "mean": [v.mean() for v in quantities.values()],
    "std/mean (%)": [100 * v.std() / v.mean() for v in quantities.values()]},
    index=list(quantities))
print(spread.round(3))
```

```
                  mean  std/mean (%)
speed            5.085         7.108
angular velocity 3.347        13.500
areal velocity   3.860         2.348
```

The speed wobbles by 7.1%, the angular velocity by 13.5%, and the areal velocity by only 2.3%. Is even that real? We checked by switching the measurement noise off: the areal velocity then wobbles by just 0.017%. So the remaining 2% is noise in the measurements, not physics.

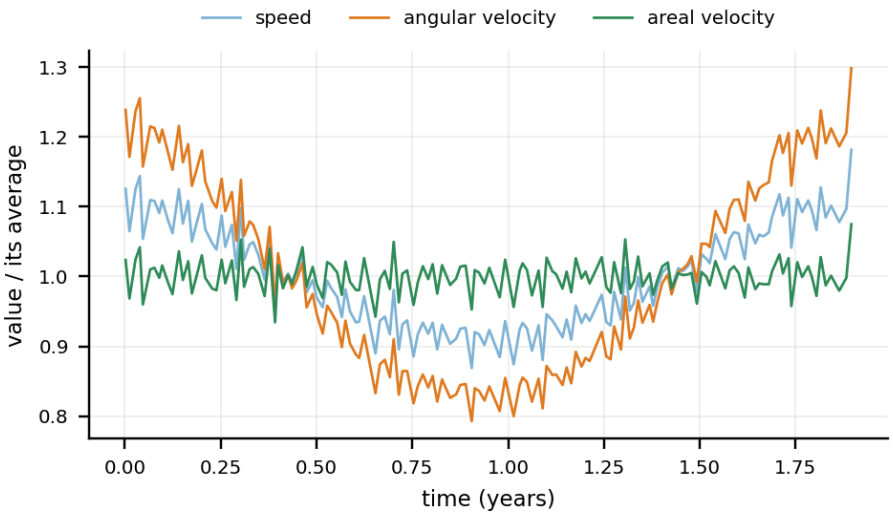


Figure 8. *Each quantity divided by its average. Only the areal velocity (green) is flat.*

Now the equal-areas picture itself. We cut the record into 0.25-year windows and add up the area swept in each (Figure 9). The wedges have very different shapes, wide near the Sun and thin far away, but the same area.

```
# Equal times -> equal areas?  Cut the record into 0.25-year windows
window = (t[:-1] // 0.25).astype(int)
swept = pd.DataFrame({"window": window, "area": area, "dt": dt})
swept = swept.groupby("window").sum()
swept = swept[swept["dt"] > 0.2]          # ignore a window cut short at the end
# rescale, in case a few cloudy nights made a window a little shorter
swept["area per 0.25 yr"] = 0.25 * swept["area"] / swept["dt"]
print(swept["area per 0.25 yr"].describe().round(4))
```

```
count    15.0000
mean      0.9649
std       0.0012
min       0.9631
25%       0.9641
50%       0.9649
75%       0.9656
max       0.9673
Name: area per 0.25 yr, dtype: float64
```

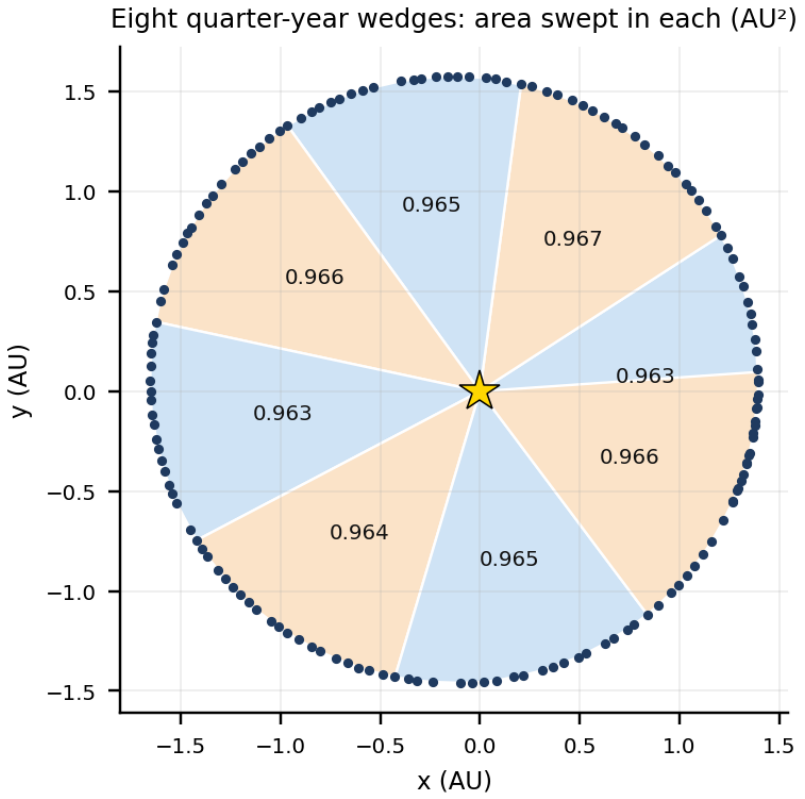


Figure 9. *Eight quarter-year wedges. The numbers are the area swept in each (AU²): practically identical.*

Over 15 windows the area is 0.965 ± 0.0012 AU² per quarter-year: a spread of just 0.13%. Wider windows average out the noise.

Let regression discover the exponent

There is a second way to see Law 2, and it uses a trick you will meet again. If the areal velocity $\frac{1}{2}r^2(d\theta/dt)$ is constant, then the angular velocity must fall as the *inverse square* of the distance. Take logarithms and any power law becomes a straight line:

$$\log(\text{angular velocity}) = k \cdot \log(r) + \text{constant}$$

Equation 2. A power law, seen through logarithms

The slope k is the exponent. If Law 2 is right, we should find $k = -2$. Let the machine tell us, and also check that the areal velocity does not drift with time.

```
from scipy import stats

fit = stats.linregress(np.log(r_mid), np.log(ang_vel))
low, high = fit.slope - 1.96 * fit.stderr, fit.slope + 1.96 * fit.stderr
```

```
print(f"log(angular velocity) = {fit.slope:.3f} * log(r) + {fit.intercept:.3f}")
print(f"slope 95% CI: [{low:.3f}, {high:.3f}]")

trend = stats.linregress(t[:-1], areal_velocity)
print(f"trend of areal velocity over time: {trend.slope:+.4f} AU^2/yr per yr "
      f"(p = {trend.pvalue:.2f})")
```

```
log(angular velocity) = -2.028 * log(r) + 2.056
slope 95% CI: [-2.070, -1.986]
trend of areal velocity over time: +0.0015 AU^2/yr per yr (p = 0.77)
```

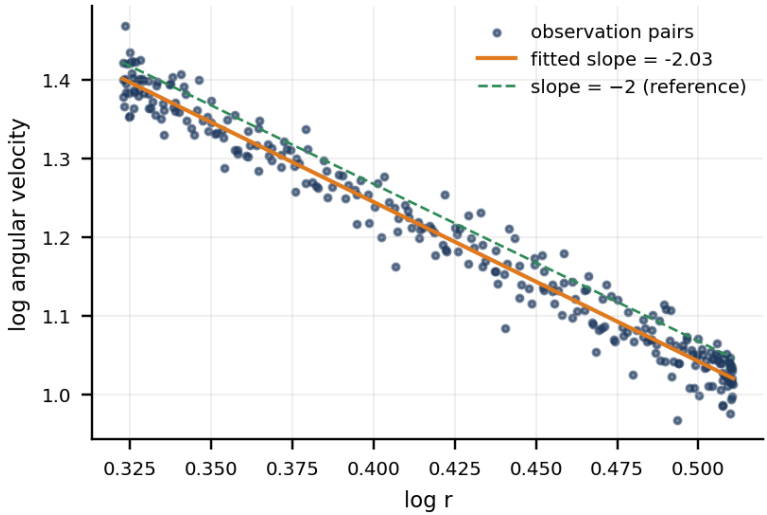


Figure 10. *Log–log plot of angular velocity against distance. The fitted slope matches −2.*

The fitted slope is -2.03, with a 95% confidence interval of [-2.07, -1.99], which contains −2. And the p-value of 0.77 says there is no trend over time: a trend this large would arise by pure chance about 77% of the time. **Law 2 holds.**

Fine-Tune Your Model: Which Law Breaks First?

Real data is noisier than ours. Let us turn up the noise and see how Laws 1 and 2 cope. (Cloudy nights are switched off here, to keep the comparison fair.)

```
def law2_spread(d):
    t, x, y = d["t"].to_numpy(), d["x"].to_numpy(), d["y"].to_numpy()
    av = 0.5 * (x[:-1] * y[1:] - x[1:] * y[:-1]) / np.diff(t)
    return 100 * av.std() / av.mean()

rows = []
for noise in [0.0, 0.001, 0.01, 0.03, 0.1]:                  # AU
    d = observe_mars(noise=noise, cloudy=0).sort_values("t")
    m = make_ellipse_model().fit(d[["x", "y"]].to_numpy(), np.hypot(d["x"], d["y"]))
    rows.append((noise, orbital_elements(m)[1], law2_spread(d)))
sweep = pd.DataFrame(rows, columns=["noise (AU)", "Law 1: e (true 0.0934)",
                                    "Law 2: spread (%)"])
print(sweep.round(4).to_string(index=False))
```

```
 noise (AU)  Law 1: e (true 0.0934)  Law 2: spread (%)
      0.000                  0.0934             0.0167
      0.001                  0.0933             2.3869
      0.010                  0.0928            23.7205
      0.030                  0.0915            70.2598
      0.100                  0.0866           223.1794
```

Law 1 is remarkably robust: even with ten times more noise than Tycho, the estimated eccentricity is within 0.01 of the truth. It fits a curve through hundreds of points, and the errors average out. **Law 2, done our way, is fragile**: it subtracts neighbouring noisy numbers and divides by a short time gap, which amplifies the noise. At 0.01 AU the areal velocity "wobbles" by 24%, and at 0.03 AU the method falls apart.

> **TIP**
> **Rule of thumb.** Methods that *fit one model through many points* tolerate noise. Methods that *subtract neighbouring points* amplify it. If you must differentiate noisy data, smooth it first, or use a longer baseline (Exercise 2).

Law 3: The Harmony of the Worlds

Kepler's third law: the square of a planet's orbital period is proportional to the cube of its average distance from the Sun, $T^2 \propto a^3$. This one links *different planets* together. Kepler found it in 1618 after a decade of searching for a harmony in the planets' numbers. We have eight planets, a very small dataset, and one chance to find the right exponent.

Discover and Visualize the Data to Gain Insights

Explore the six training planets only. Add the combined column T^2/a^3, which is the type of attribute combination the book recommends, and logarithms, because the numbers are tail-heavy.

```
eda = train_planets.copy()
eda["T2_over_a3"] = eda["T_years"] ** 2 / eda["a_au"] ** 3
eda["log_a"], eda["log_T"] = np.log(eda["a_au"]), np.log(eda["T_years"])
print(eda.round(4).to_string(index=False))
print("corr(T, a)         =", round(eda["T_years"].corr(eda["a_au"]), 4))
print("corr(log T, log a) =", round(eda["log_T"].corr(eda["log_a"]), 6))
```

```
   name   a_au  T_years  T2_over_a3   log_a   log_T
Mercury 0.3871   0.2408      1.0001 -0.9491 -1.4236
  Venus 0.7233   0.6152      1.0000 -0.3239 -0.4858
  Earth 1.0000   1.0000      1.0000  0.0000  0.0000
   Mars 1.5237   1.8808      1.0000  0.4211  0.6317
Jupiter 5.2029  11.8620      0.9990  1.6492  2.4733
 Saturn 9.5367  29.4571      1.0004  2.2551  3.3829
corr(T, a)         = 0.9924
corr(log T, log a) = 1.0
```

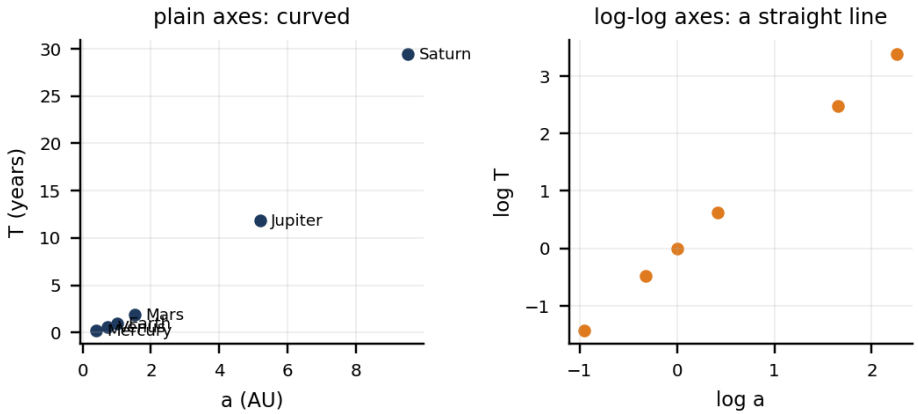


Figure 11. *On plain axes the planets lie on a curve. On log–log axes they lie on a straight line.*

The ordinary correlation between T and a is a respectable 0.992, but the plot is clearly curved. The correlation between *log T* and *log a* is **1.000000**. The combination T^2/a^3 is 1.000 for every planet, to within 0.1%. Hey, not bad. But do not celebrate: you guessed the exponents 2 and 3. Let us make the machine find them, and prove it on planets it has never seen.

Prepare the Data for Machine Learning Algorithms

No cleaning is needed. The only transformation is the logarithm. In Scikit-Learn terms: `FunctionTransformer(np.log)` for the input, and `TransformedTargetRegressor` to take the log of the target and undo it for predictions.

Select and Train a Model

Same game as before: four models, from too simple to too flexible. With six planets, the cross-validation splits each hold out just one planet at a time (**leave-one-out**), and we also report the errors on the locked-away Uranus and Neptune, as percentages.

```
from sklearn.model_selection import LeaveOneOut, cross_val_predict
from sklearn.preprocessing import FunctionTransformer

a_tr, T_tr = train_planets[["a_au"]].to_numpy(), train_planets["T_years"].to_numpy()
a_te, T_te = test_planets[["a_au"]].to_numpy(), test_planets["T_years"].to_numpy()

def pct_error(true, pred):
    return 100 * np.abs(pred - true) / true

power_law = TransformedTargetRegressor(
    regressor=make_pipeline(FunctionTransformer(np.log), LinearRegression()),
    func=np.log, inverse_func=np.exp)

models3 = {
```

```
    "Straight line  T ~ a": LinearRegression(),
    "Decision tree": DecisionTreeRegressor(random_state=42),
    "Random forest": RandomForestRegressor(random_state=42),
    "Power law  log T ~ log a": power_law,
}
rows = []
for name, model in models3.items():
    # leave-one-out: hide one planet at a time (6 folds for 6 planets)
    loo = cross_val_predict(model, a_tr, T_tr, cv=LeaveOneOut())
    model.fit(a_tr, T_tr)
    test_err = pct_error(T_te, model.predict(a_te))
    rows.append((name, pct_error(T_tr, loo).mean(), *test_err))
cols = ["model", "LOO CV error %", "Uranus err %", "Neptune err %"]
results3 = pd.DataFrame(rows, columns=cols).set_index("model")
print(results3.round(2))
```

```
                          LOO CV error %  Uranus err %  Neptune err %
model
Straight line  T ~ a              155.58         30.03          43.33
Decision tree                     74.53         64.94          82.12
Random forest                     86.13         72.58          86.02
Power law  log T ~ log a           0.02          0.03           0.05
```

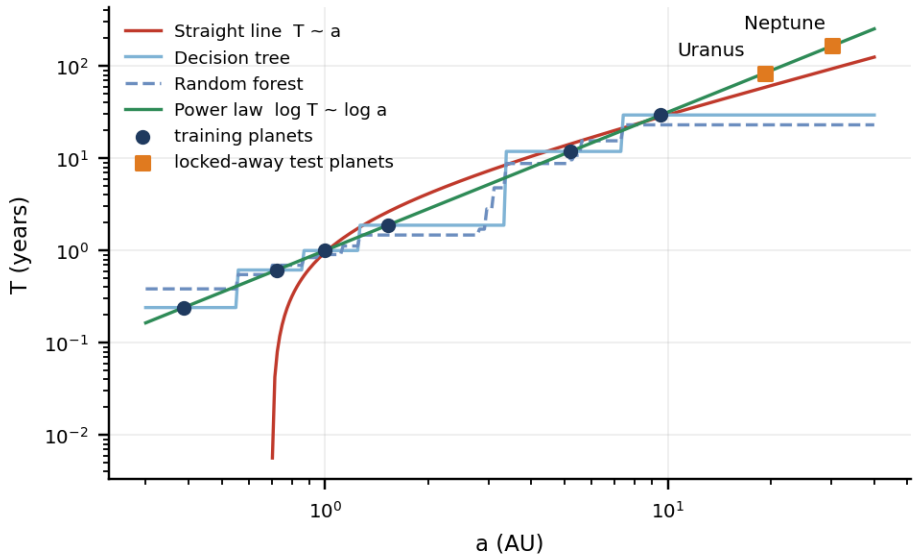


Figure 12. *Fitted on six planets, asked about two it never saw. Trees and forests flatten out beyond Saturn.*

The results are dramatic. The straight line is off by 156% in cross-validation (it even predicts negative years for tiny orbits). The tree and the forest are off by 75 to 86%, and by 65 to 86% for Uranus and Neptune: they can only repeat what they saw, so for any planet beyond Saturn they answer “about 29 years”. The **power law** is off by 0.02% in cross-validation, and by just 0.03% and 0.05% for Uranus and Neptune. **It predicted the periods of two planets it had never seen, to within a twentieth of a percent.**

Fine-Tune Your Model

The log–log regression found the exponent for us, but let us confirm it a second way, in the spirit of the book's grid search. Try every exponent p from 0.5 to 2.5 in steps of 0.01. For each, ask: how constant is T / a^p across the planets? The right p makes it as constant as possible.

```
exponents = np.arange(0.5, 2.501, 0.01)
# how "constant" is T / a^p across the planets?  (0 = perfectly constant)
spread3 = [np.std(np.log(T_tr / a_tr[:, 0] ** p)) for p in exponents]
best_p = exponents[np.argmin(spread3)]
print(f"best exponent p = {best_p:.2f}")
```

```
best exponent p = 1.50
```

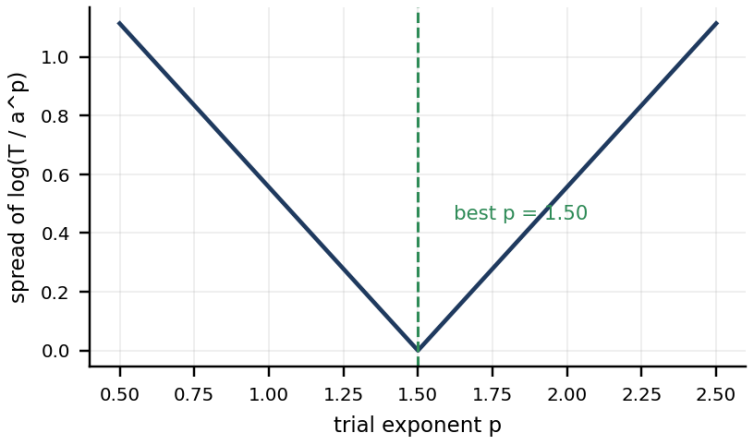


Figure 13. *Grid search over the exponent. The valley is sharp and sits at exactly 1.5.*

Now the regression, with a confidence interval for the slope, and the constant T^2/a^3 for all eight planets.

```
fit3 = stats.linregress(np.log(a_tr[:, 0]), np.log(T_tr))
t_crit = stats.t.ppf(0.975, df=len(T_tr) - 2)
half = t_crit * fit3.stderr
print(f"log T = {fit3.slope:.4f} * log a + {fit3.intercept:.4f}")
print(f"slope 95% CI: [{fit3.slope - half:.4f}, {fit3.slope + half:.4f}]")

K = planets["T_years"] ** 2 / planets["a_au"] ** 3
print("T^2 / a^3 for all eight planets:", K.round(4).tolist())
```

```
log T = 1.5000 * log a + -0.0000
slope 95% CI: [1.4997, 1.5002]
T^2 / a^3 for all eight planets: [1.0001, 1.0, 1.0, 1.0, 0.999, 1.0004, 0.9991, 0.9988]
```

The exponent is 1.5000, with a 95% confidence interval of [1.4997, 1.5002]: **3/2 exactly**, that is, $T^2 \propto a^3$. The constant is 1 because we measure in years and AU; in kilometres and seconds it would be $4\pi^2/(GM)$.

NOTE

Jupiter's ratio is 0.9990, not 1.0000. That tiny deviation is real physics, not error: Jupiter is heavy enough to tug the Sun noticeably, and the exact version of the law is $T^2 = 4\pi^2 a^3 / (G(M + m))$, with M the Sun and m the planet.

Present Your Solution

Before launch, present what you learned: what worked, what did not, the assumptions, and the limitations. Start with a one-page summary Kepler can pin to his wall.

Law	What the data say	Evidence
1. Ellipse	Orbit is an ellipse, Sun at a focus. a = 1.5239 AU [1.5237, 1.5240], e = 0.0934 [0.0933, 0.0935], ω = 336.00°	Test RMSE 0.00099 AU = noise level; works when learning from half the orbit
2. Equal areas	Areal velocity ½ r^2 dθ/dt constant (≈ 3.86 AU^2/yr). Log–log slope -2.03 [-2.07, -1.99]	Quarter-year areas agree to 0.13%; no drift (p = 0.77)
3. Harmonies	$T \propto a^{1.5000}$ [1.4997, 1.5002], i.e. $T^2 = a^3$ (years, AU)	Uranus and Neptune predicted to within 0.03% and 0.05%

Easy-to-remember statements (the book recommends them):

- *Flip the distance (1/r) and the ellipse becomes a straight line in cos and sin.*
- *Equal areas in equal times means r^2 times the angular speed never changes.*
- *Take logs and the planets line up on a ruler of slope 3/2.*

THE BIG IDEA

The big idea behind all three laws: transform the problem until it becomes a **straight line**, then let ordinary linear regression do the rest. No neural networks, no heavy computing: three small regressions found three laws that took Kepler years of hand calculation. That is the most efficient route, and also the easiest to explain.

The reveal

Now we open the sealed envelope. Mars' true orbit, which we fed into the simulation, has a = 1.52371 AU, e = 0.0934 and ω = 336.0°. You found 1.5239, 0.0934 and 336.00°, and all three true values fall inside your 95% bootstrap intervals. You rediscovered the orbit from noisy data without ever being told it.

What worked, what did not

- **Worked:** feature engineering (1/r, cos, sin, logs) + `LinearRegression`; honest test sets; residual plots; cross-validation.

- **Did not:** trees and forests, which memorise or flatten out; and the neighbour-difference method for Law 2, which breaks as noise grows.
- **Assumptions:** Sun-centred positions (the real Kepler had to compute them), one flat plane, only the Sun's gravity.
- **Limitations:** the Mars data are simulated; Law 3 has only eight planets, two of them in the test set; the Sun-only model ignores small effects such as other planets' pulls (the 0.1% wobble above) and relativity (the tiny slow turning of Mercury's orbit).

> **WHY DO THESE LAWS HOLD?**
> Kepler published the first two laws in 1609 and the third in 1619. In 1687 Isaac Newton showed that all three follow from one rule: gravity pulls with a force that falls with the *square* of the distance. Equal areas hold for any force pointing at the Sun; the ellipse and the exponent 3/2 are specific to the inverse-square law. That is why a simulation that knew only Newton's gravity could produce data that obeys all three of Kepler's.

Launch, Monitor, and Maintain Your System

Approval granted! To get ready for production, wrap what you learned into small reusable functions, and save trained models so that you can load them later:

```
import joblib
joblib.dump(final_model, "mars_orbit.pkl")      # save the trained model
final_model = joblib.load("mars_orbit.pkl")     # ...and load it later
```

Here is a tiny production version of Law 3, plus a **monitor** that raises an alert when predictions drift out of tolerance:

```
def predict_period(a_au, K=1.0):
    """Kepler's third law: T^2 = K * a^3   (T in years, a in AU)."""
    return np.sqrt(K * np.asarray(a_au) ** 3)

def check_health(true, pred, tolerance_pct=1.0):
    err = 100 * abs(pred - true) / true
    verdict = "OK" if err <= tolerance_pct else "ALERT: re-calibrate the model"
    print(f"error = {err:6.2f} %  ->  {verdict}")
```

Test it on a newcomer that is not even a planet: **Halley's comet**, a dirty snowball on a stretched orbit.

```
>>> check_health(true=75.32, pred=predict_period(17.834))
error =   0.01 %  ->  OK
```

Off by one hundredth of a percent. The law generalises. Now feed the same function a different kind of system: the four large moons of Jupiter, which orbit Jupiter, not the Sun.

```
AU_KM, YEAR_DAYS = 149_597_870.7, 365.25
moons = pd.DataFrame({
    "name": ["Io", "Europa", "Ganymede", "Callisto"],
    "a_km": [421_700, 671_034, 1_070_412, 1_882_709],
    "T_days": [1.769138, 3.551181, 7.154553, 16.689018]})
moons["a_au"] = moons["a_km"] / AU_KM
moons["T_years"] = moons["T_days"] / YEAR_DAYS

print("The Sun's law, applied to Jupiter's moons:")
for _, m in moons.iterrows():
    check_health(m["T_years"], predict_period(m["a_au"]))

slope = stats.linregress(np.log(moons["a_au"]), np.log(moons["T_years"])).slope
K_jup = (moons["T_years"] ** 2 / moons["a_au"] ** 3).mean()
print(f"\nRe-fitted exponent for the moons: {slope:.3f}  (the SHAPE survives)")
print(f"New constant K = {K_jup:.0f} yr^2/AU^3 -> Jupiter is 1/{K_jup:.0f} of the Sun")
```

```
The Sun's law, applied to Jupiter's moons:
error =  96.91 %  ->  ALERT: re-calibrate the model
error =  96.91 %  ->  ALERT: re-calibrate the model
error =  96.91 %  ->  ALERT: re-calibrate the model
error =  96.91 %  ->  ALERT: re-calibrate the model

Re-fitted exponent for the moons: 1.500  (the SHAPE survives)
New constant K = 1047 yr^2/AU^3 -> Jupiter is 1/1047 of the Sun
```

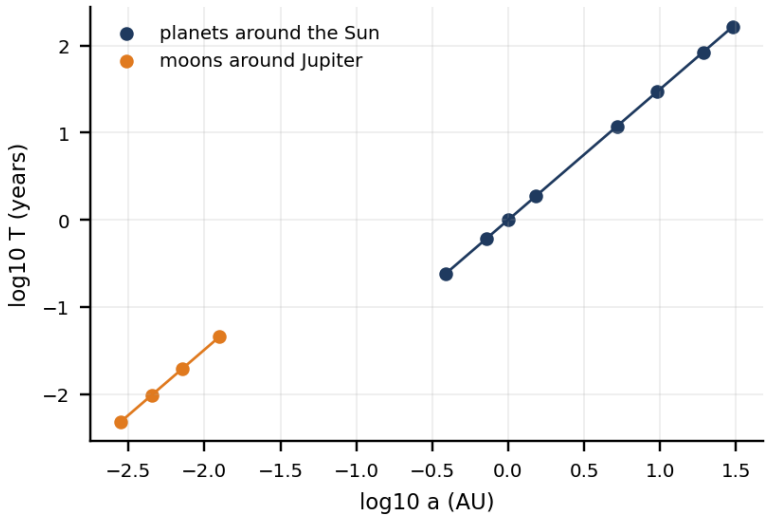


Figure 14. *Same slope (3/2), different intercept: the planets circle the Sun, the moons circle Jupiter.*

The monitor fires, as it should: the Sun-calibrated constant is wrong for another central body, every moon is off by the same 97%. This is **model rot** in action, the effect the book warns about. But look at the fix. The *shape* of the law survives (slope 1.500); only the constant K must be re-calibrated, and the new K carries a bonus. Since K is inversely proportional to the central mass, K ≈ 1047 says **Jupiter weighs about 1/1047 of the Sun**. That is correct: Jupiter is just under a thousandth of a solar mass. You have just weighed a planet using four moons and a ruler.

Monitoring and maintenance checklist

- **Watch the inputs.** Count cloudy nights (missing values), check units, and flag impossible values early.
- **Watch the residuals.** Re-plot Figure 5 on fresh data regularly. If structure creeps back in (waves instead of noise), your law is missing something.
- **Re-fit on a schedule.** The script runs in 10 seconds; run it whenever new observations arrive. Save every model with its scores, so you can compare and roll back.
- **Keep the test set sacred.** Never tune on it, or it stops being a test.

Try It Out!

Hopefully this guide gave you a good idea of what an ML project looks like. The best way to learn is to take the code and break it. Here are some exercises, from easy to hard.

1. **Noise.** Re-run with `noise=0.01`. Which law breaks first? (Answer in the table of Law 2.)
2. **Make Law 2 robust.** Compute the areal velocity from observations k steps apart (try $k = 5, 10$) or over the 0.25-year windows. How does the spread change with noise 0.01?
3. **Blunders.** Add 10 observations with errors of 0.1 AU. Replace `LinearRegression` by `HuberRegressor` (or `RANSACRegressor`). How do a, e and ω change?
4. **Optimisation.** Refine the ellipse directly in x–y space with `scipy.optimize.least_squares`, starting from the regression answer. Is it better, and by how much?
5. **Real data.** Download real heliocentric positions of Mars from NASA's JPL Horizons service and re-run Laws 1 and 2. (We have not tested this step in this guide: expect to adapt the data-loading code, and check units and reference frame.)
6. **Derive it.** A conic is the set of points whose distance to the focus equals e times the distance to a straight line (the directrix). Use that to derive $r = p/(1 + e \cos \theta)$.
7. **Other worlds.** Generalise `observe_mars` to accept any a and e. Fit Mercury ($e \approx 0.2056$) and Halley's comet ($e \approx 0.967$). Where does the method struggle?
8. **RandomizedSearchCV.** Replace the exponent grid by a random search, as the book does for hyperparameters.
9. **Symbolic regression.** Install PySR or a similar library and ask it to discover T = f(a) from the six training planets. Does it find $a^{1.5}$ by itself?

Appendix: Kepler's Laws on One Page

Law	Statement	Formula
1	Planets move on ellipses, with the Sun at one focus.	$r(\theta) = p / (1 + e\cos(\theta - \omega))$, $p = a(1 - e^2)$
2	The line from the Sun to a planet sweeps out equal areas in equal times.	$dA/dt = \frac{1}{2} r^2 d\theta/dt$ = constant
3	The square of the orbital period is proportional to the cube of the semi-major axis.	$T^2 = K a^3$ ($K = 1$ for years, AU, the Sun)

Words you met

Word	Meaning
AU	Astronomical unit, the Earth–Sun distance, about 150 million km.
Semi-major axis a	Half of the ellipse's long diameter.
Eccentricity e	How squashed the ellipse is: 0 is a circle, near 1 is a long thin one.
Focus (plural: foci)	One of two special points inside an ellipse. The Sun is at one; the other is empty.
Perihelion / aphelion	The point of the orbit closest to / farthest from the Sun.
Regression	Predicting a number from other numbers.
RMSE	Root Mean Square Error: the typical size of a prediction error.
Overfitting / underfitting	Memorising noise (great on training data, bad on new data) / being too simple to capture the pattern.
Cross-validation	Estimating performance by repeatedly hiding part of the training data.
Hyperparameter	A setting you choose before training (for example, the number of harmonics).
Pipeline	A chain of data-preparation steps and a model, run as one object.
Bootstrap	Re-sampling your data many times to see how much an estimate moves.

Credit. The structure and teaching approach of this guide follow Chapter 2 of *Hands-On Machine Learning with Scikit-Learn and TensorFlow* by Aurélien Géron (O'Reilly). All text, code and data of this guide are original to it.